

IT6130 — Lập trình và ảo hóa cho IoT

MỘT SỐ ĐỀ TÀI PROJECT (Phần 2)

5 Project — Hệ thống IoT ảo hóa

Kiến trúc lai: ThingsBoard Cloud + Edge Docker Stack

Programming and Virtualization for IoT

Đồ án nhóm · Mỗi nhóm 3 sinh viên · Báo cáo + Demo + Vấn đáp

Công cụ chính: Docker, Docker Compose, Python (paho-mqtt, FastAPI), Mosquitto MQTT, InfluxDB, Grafana, **ThingsBoard** (Cloud/Edge), REST API, JWT.

Năm thành phần sinh viên phải tự lập trình: virtual sensor, virtual actuator, edge IoT gateway, rule engine và REST API.

Mục lục

Khung yêu cầu chung	2
1 Đề tài 1 — Virtual Smart Healthcare Ward Gateway	5
2 Đề tài 2 — Virtual Smart Parking & Traffic Gateway	11
3 Đề tài 3 — Virtual Smart Water & Flood Early-Warning Gateway	16
4 Đề tài 4 — Virtual Smart Air Quality Gateway	21
5 Đề tài 5 — Virtual Smart Waste Management Gateway	26

Khung yêu cầu chung cho các đề tài

Năm đề tài trong tài liệu này dùng chung một khung kiến trúc và một bộ yêu cầu nền tảng. Mỗi nhóm chọn một đề tài, nhưng tất cả đều phải tuân thủ khung chung dưới đây. Phần riêng của từng đề tài (bối cảnh, cảm biến, luật xử lý, dashboard, tiêu chí chấm điểm...) được trình bày đầy đủ trong từng đề tài.

1. Mục tiêu chung

Mỗi nhóm xây dựng một **hệ thống IoT hoàn chỉnh ở mức phần mềm**, trong đó toàn bộ thiết bị vật lý (sensor, actuator, gateway) được *giả lập* bằng chương trình Python chạy trong container Docker. Hệ thống phải thể hiện được cả hai chiều dữ liệu đã học: chiều *uplink* (telemetry từ thiết bị lên cloud) và chiều *downlink* (lệnh điều khiển RPC từ cloud xuống thiết bị).

Điểm khác biệt cốt lõi so với labwork cơ bản: hệ thống dùng **kiến trúc lai (hybrid)**. Một **Edge IoT Gateway** tự lập trình đứng giữa, vừa xử lý dữ liệu cục bộ tại biên (normalize, rule engine, lưu InfluxDB, dashboard Grafana), vừa đóng vai trò **ThingsBoard Gateway** đẩy dữ liệu nhiều thiết bị con lên **ThingsBoard Cloud** và nhận lệnh RPC từ cloud để điều khiển actuator.

2. Kiến trúc lai bắt buộc

Mọi đề tài đều theo mô hình sau (chi tiết hóa theo từng lĩnh vực ở mỗi đề tài):

1. **Virtual sensor** (Python/Java/C..., container): mỗi node sinh telemetry định kỳ, publish JSON lên Mosquitto broker cục bộ.
2. **Virtual actuator** (Python/Java/C..., container): subscribe topic lệnh, cập nhật trạng thái, publish lại status.
3. **Mosquitto MQTT broker** (container): trục giao tiếp nội bộ tại edge.
4. **Edge IoT Gateway** (Python/Java/C..., container) — thành phần quan trọng nhất:
 - Subscribe toàn bộ telemetry từ các node; validate và normalize dữ liệu.
 - Chạy **rule engine** (≥ 4 luật), sinh event bất thường, gửi command tự động xuống actuator.
 - Ghi telemetry/event/status vào **InfluxDB**.
 - Hoạt động như **ThingsBoard Gateway**: connect các sub-device, đẩy telemetry lên ThingsBoard, nhận RPC từ ThingsBoard và chuyển thành command MQTT cục bộ.
5. **InfluxDB** (container): time-series database tại edge.
6. **Grafana** (container): dashboard giám sát đọc dữ liệu từ InfluxDB.
7. **FastAPI REST API** (Python/Java/C..., container): truy vấn trạng thái và gửi lệnh thủ công; tham chiếu được tới ThingsBoard REST API (đăng nhập JWT).
8. **ThingsBoard Cloud**: nền tảng IoT đám mây — dashboard, Rule Chain, Alarm theo ngưỡng và RPC điều khiển từ xa.

3. ThingsBoard Gateway API (dùng chung)

Gateway kết nối ThingsBoard bằng MQTT với username = ACCESS_TOKEN của một thiết bị kiểu *Gateway*. Các topic chuẩn (đã học mở rộng từ Buổi 1–2):

```
# Announce a sub-device connection (it then appears on ThingsBoard)
publish v1/gateway/connect {"device": "bed-01"}
# Push telemetry for MANY sub-devices at once
publish v1/gateway/telemetry {"bed-01": [{"ts": 1750000000000,
    "values": {"heart_rate": 78, "spo2": 97.5}}]}
# Receive an RPC command from server for a sub-device
subscribe v1/gateway/rpc {"device": "bed-01",
    "data": {"id": 1, "method": "setNurseCall", "params": true}}
# Return the RPC result back to server
publish v1/gateway/rpc {"device": "bed-01", "id": 1, "data": {"success": true}}
```

Liên hệ với kiến thức đã học

Đây chính là mở rộng tự nhiên của Buổi 1 (telemetry qua `v1/devices/me/telemetry`) và Buổi 2 (RPC qua `v1/devices/me/rpc/request/+`). Khác biệt: thay vì 1 thiết bị tự kết nối, *gateway* đại diện cho nhiều thiết bị con, giúp giảm số kết nối và xử lý trước tại biên — đúng tinh thần edge computing.

4. Yêu cầu triển khai Docker (dùng chung)

Bắt buộc với mọi đề tài

- Toàn bộ hệ thống chạy được bằng một lệnh: `docker compose up -d --build`.
- Mỗi service tự lập trình phải có `Dockerfile` riêng.
- Cấu hình đọc từ **environment variables**, không hard-code trong source.
- Các service gọi nhau bằng **service name** (vd `mosquitto`, `http://influxdb:8086`), tuyệt đối không dùng `localhost` để gọi service khác.
- Dùng **Docker volume** cho InfluxDB và Grafana; khuyến khích volume/bind mount cho cấu hình Mosquitto.
- Các service giao tiếp qua một Docker Compose network chung.

Biến môi trường tối thiểu (đặt trong `.env`, không commit token thật):

```
MQTT_BROKER=mosquitto
MQTT_PORT=1883
PUBLISH_INTERVAL=2
INFLUXDB_URL=http://influxdb:8086
INFLUXDB_TOKEN=...      INFLUXDB_ORG=hust      INFLUXDB_BUCKET=iot
TB_HOST=thingsboard.cloud TB_PORT=1883      TB_GATEWAY_TOKEN=... # Gateway device token
```

5. Cấu trúc thư mục gợi ý (dùng chung)

```
project-xx/
|-- docker-compose.yml      |-- README.md          |-- .env.example
|-- report.pdf              |-- mosquitto/config/mosquitto.conf
|-- virtual_sensor/         (sensor.py, requirements.txt, Dockerfile)
|-- virtual_actuator/       (actuator.py, requirements.txt, Dockerfile)
|-- iot_gateway/            (gateway.py, rule_engine.py, tb_gateway.py,
|                           state_store.py, requirements.txt, Dockerfile)
|-- gateway_api/            (api.py, requirements.txt, Dockerfile)
|-- grafana/                (provisioning/, dashboards/)
|-- docs/                   (architecture.png, topic-design.md, thingsboard-setup.md)
`-- screenshots/            (compose-ps, mqtt, influxdb, grafana, thingsboard-dashboard)
```

6. Yêu cầu báo cáo, README và demo (dùng chung)

Mỗi nhóm nộp source code, `docker-compose.yml`, các `Dockerfile`, `README.md`, báo cáo PDF khoảng 15–20 trang và ảnh chụp màn hình. Báo cáo trình bày: kiến trúc, thiết kế topic/message, logic xử lý, rule engine, tích hợp ThingsBoard (uplink + RPC), triển khai Docker Compose, dashboard Grafana và ThingsBoard, phân công công việc.

`README.md` phải đủ để người khác chạy lại: mô tả hệ thống, sơ đồ luồng dữ liệu, danh sách service, cách chạy, cách xem log, cách truy cập Grafana/InfluxDB/ ThingsBoard/REST API, cách gửi lệnh thử công và các lỗi thường gặp. Các lệnh tối thiểu: `docker compose up -d --build`, `docker compose ps`, `docker compose logs -f iot-gateway`, `docker compose down`.

Khi demo, nhóm phải chứng minh: (1) chạy toàn bộ stack bằng Docker Compose; (2) nhiều virtual sensor đang publish; (3) gateway nhận telemetry, normalize và phát hiện ít nhất một event bất thường; (4) gateway

gửi command tự động tới actuator và actuator publish lại status; (5) dữ liệu vào InfluxDB; (6) Grafana hiển thị telemetry/event/status; (7) telemetry xuất hiện trên ThingsBoard; (8) gửi RPC từ ThingsBoard (hoặc REST API) xuống actuator thành công.

7. Ràng buộc chung

Ràng buộc áp dụng cho tất cả đề tài

- Không yêu cầu mua sensor/gateway/actuator thật; không dùng dịch vụ cloud trả phí.
- Dùng ThingsBoard miễn phí: `thingsboard.cloud` (demo) hoặc bản tự cài (CE).
- Hệ thống phải chạy được trên laptop cá nhân, các service chính chạy bằng Docker Compose.
- Mỗi nhóm phải tự lập trình tối thiểu: virtual sensor, virtual actuator, edge IoT gateway và REST API; gateway phải có thêm logic rule engine, tích hợp ThingsBoard và command/control — không chỉ sửa lại labwork.
- Không lộ ACCESS_TOKEN/JWT trong source công khai; cấu hình qua `.env`.

Đề tài 1

Virtual Smart Healthcare Ward Gateway

1

1 Đề tài 1 — Virtual Smart Healthcare Ward Gateway

1.1 Tên đề tài

Xây dựng hệ thống **Virtual IoT Gateway giám sát bệnh nhân trong khoa điều trị**, có khả năng thu thập sinh hiệu (vital signs), chuẩn hóa dữ liệu, phát hiện tình huống nguy cấp bằng rule engine, tự động kích hoạt chuông gọi y tá và điều khiển thiết bị qua MQTT, đồng thời đẩy dữ liệu lên ThingsBoard Cloud và nhận lệnh RPC điều khiển từ xa.

1.2 Bối cảnh

Trong bệnh viện thông minh, mỗi giường bệnh được trang bị các cảm biến đeo hoặc đặt cạnh giường để theo dõi liên tục nhịp tim, nồng độ oxy máu (SpO_2), nhiệt độ cơ thể, nhịp thở và huyết áp. Việc theo dõi thủ công nhiều giường cùng lúc rất dễ bỏ sót thời điểm bệnh nhân chuyển nặng. Một *edge gateway* đặt tại khoa có thể thu thập dữ liệu từ nhiều giường, phát hiện ngay các ngưỡng nguy hiểm (nhịp tim quá cao/thấp, SpO_2 tụt, sốt cao, té ngã) và lập tức cảnh báo cho điều dưỡng mà không phụ thuộc vào đường truyền lên cloud.

Mini-project yêu cầu sinh viên xây dựng một hệ thống hoàn chỉnh ở mức phần mềm: toàn bộ cảm biến sinh hiệu, thiết bị cảnh báo và gateway đều được giả lập bằng Python trong container Docker. Hệ thống vừa giám sát/điều khiển cục bộ tại biên, vừa đồng bộ với ThingsBoard Cloud để hiển thị và điều khiển từ xa.

Lưu ý phạm vi

Đây là bài toán *mô phỏng phục vụ học tập*. Các ngưỡng và logic chỉ minh họa kỹ thuật IoT, không phải khuyến cáo y khoa và không dùng cho mục đích lâm sàng.

1.3 Mục tiêu của mini-project

Sau khi hoàn thành, nhóm sinh viên cần đạt được:

1. Lập trình nhiều virtual sensor node mô phỏng sinh hiệu của các giường bệnh khác nhau.
2. Xây dựng edge gateway nhận telemetry MQTT từ nhiều giường.
3. Thiết kế message format và topic hierarchy hợp lý cho dữ liệu y tế.
4. Chuẩn hóa dữ liệu thô (đơn vị, khoảng giá trị, gắn nhãn thời gian) về một định dạng thống nhất.
5. Phát hiện tình huống nguy cấp bằng rule engine có phân mức độ (info/warning/critical).
6. Tự động gửi lệnh điều khiển (gọi y tá, bật van oxy) tới actuator qua MQTT.
7. Lưu sinh hiệu, event và trạng thái actuator vào InfluxDB.
8. Tích hợp ThingsBoard: đẩy telemetry nhiều giường qua `v1/gateway/telemetry` và nhận RPC qua `v1/gateway/rpc`.
9. Xây dựng dashboard giám sát trên cả Grafana (edge) và ThingsBoard (cloud).
10. Cung cấp REST API truy vấn trạng thái giường và gửi lệnh thủ công.
11. Container hóa toàn bộ và triển khai bằng Docker Compose.

1.4 Mô tả bài toán

Hệ thống mô phỏng một khoa (ward-A) gồm tối thiểu 3 giường: bed-01, bed-02, bed-03. Mỗi giường có:

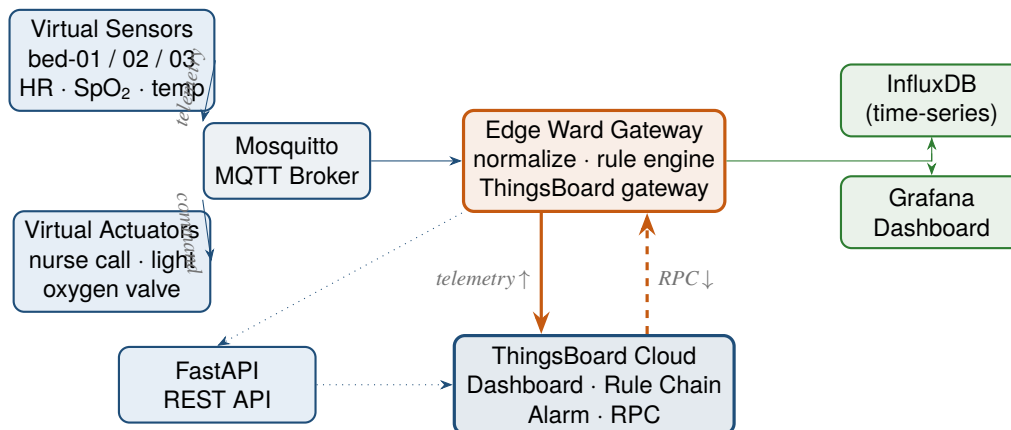
- Một **virtual sensor node** gửi định kỳ: nhịp tim, SpO_2 , nhiệt độ cơ thể, nhịp thở, huyết áp tâm thu/tâm trương, trạng thái có bệnh nhân trên giường, cờ phát hiện té ngã.

- Một **virtual actuator node** gồm: chuông gọi y tá (nurse_call), đèn đầu giường (light), van oxy mô phỏng (oxygen).

Edge gateway nhận dữ liệu từ các sensor node, xử lý và gửi lệnh điều khiển tới actuator khi phát hiện nguy cấp; đồng thời đồng bộ dữ liệu hai chiều với ThingsBoard.

1.5 Kiến trúc hệ thống yêu cầu

Kiến trúc tối thiểu như Hình 1.5.



Hình 1: Kiến trúc lai của Đề tài 1: edge gateway xử lý cục bộ và đồng bộ hai chiều với ThingsBoard Cloud.

1.6 Các service bắt buộc

File docker-compose.yml phải triển khai tối thiểu:

- mosquitto — MQTT broker.
- sensor-bed-01/02/03 — sensor mỗi giường.
- actuator-bed-01/02/03 — actuator mỗi giường.
- ward-gateway — edge IoT gateway.
- influxdb — time-series database.
- grafana — dashboard edge.
- ward-api — REST API (FastAPI).

Nhóm có thể gộp/tách service tùy thiết kế nhưng phải đủ các nhóm chức năng: sensor simulator, actuator simulator, MQTT broker, gateway, database, dashboard, API và phải giải thích trong báo cáo.

1.7 Thiết kế MQTT topic

Thiết kế topic hierarchy rõ ràng theo khoa/giường:

```

ward-A/bed-01/sensor/telemetry      # raw vital signs from sensor
ward-A/bed-01/actuator/command      # control command down to actuator
ward-A/bed-01/actuator/status       # actuator status feedback
ward-A/bed-01/gateway/normalized    # normalized data
ward-A/bed-01/gateway/event         # critical event raised by gateway
  
```

1.8 Message format yêu cầu

Telemetry từ sensor:

```

{
  "device_id": "sensor-bed-01", "ward_id": "ward-A", "bed_id": "bed-01",
  "heart_rate": 78, "spo2": 97.5, "body_temp": 36.8,
  "respiration_rate": 16, "systolic_bp": 118, "diastolic_bp": 76,
  "bed_occupancy": true, "fall_detected": false,
}
  
```

```
"timestamp": "2026-06-10T10:00:00Z"
}
```

Command từ gateway tới actuator ($\text{target} \in \{\text{nurse_call}, \text{light}, \text{oxygen}\}$, $\text{action} \in \{\text{on}, \text{off}\}$):

```
{ "bed_id": "bed-01", "target": "nurse_call", "action": "on",
  "reason": "spo2_low", "timestamp": "2026-06-10T10:00:05Z" }
```

Status từ actuator:

```
{ "device_id": "actuator-bed-01", "bed_id": "bed-01",
  "nurse_call": "on", "light": "off", "oxygen": "on",
  "last_command_reason": "spo2_low", "timestamp": "2026-06-10T10:00:06Z" }
```

Event từ gateway:

```
{ "bed_id": "bed-01", "event_type": "spo2_low", "severity": "critical",
  "value": 88.0, "threshold": 92.0, "action_taken": "nurse_call_on,oxygen_on",
  "timestamp": "2026-06-10T10:00:05Z" }
```

1.9 Yêu cầu lập trình

Virtual sensor

1. Đọc cấu hình từ env: WARD_ID, BED_ID, DEVICE_ID, MQTT_BROKER, PUBLISH_INTERVAL.
2. Sinh sinh hiệu biến đổi theo thời gian quanh giá trị nền sinh lý (không random hoàn toàn độc lập): lưu giá trị trước, thay đổi nhẹ mỗi chu kỳ.
3. Sinh tình huống bất thường theo xác suất/chu kỳ: SpO₂ tụt, nhịp tim tăng vọt, sốt cao, hoặc fall_detected = true.
4. Publish JSON lên ward-A/<bed>/sensor/telemetry.

Virtual actuator

1. Subscribe topic command của giường tương ứng; parse JSON.
2. Cập nhật trạng thái nội bộ nurse_call, light, oxygen.
3. Publish lại trạng thái lên topic status; ghi log lệnh hợp lệ/sai định dạng.

Edge IoT gateway

Thành phần quan trọng nhất, phải:

1. Subscribe telemetry tất cả giường; validate bed_id, timestamp, các field.
2. Chuẩn hóa message về format thống nhất; lưu trạng thái mới nhất từng giường.
3. Ghi telemetry vào InfluxDB; chạy rule engine; sinh và publish event; ghi event vào InfluxDB.
4. Gửi command tự động tới actuator khi cần.
5. Đẩy telemetry các giường lên ThingsBoard (mục 1.10) và nhận RPC từ ThingsBoard.

Rule engine

Tối thiểu 4 luật (nhóm bổ sung thêm được):

1. heart_rate > 120 hoặc < 50 $\Rightarrow$ nurse_call=on, event hr_abnormal, severity critical.
2. spo2 < 92 $\Rightarrow$ oxygen=on và nurse_call=on; nếu < 88 severity critical, ngược lại warning.
3. body_temp > 38.5 $\Rightarrow$ event fever, nurse_call=on, severity warning.
4. fall_detected = true $\Rightarrow$ nurse_call=on, event fall, severity critical.
5. (Mở rộng) bed_occupancy=false mà oxygen=on $\Rightarrow$ event device_running_no_patient; sensor không gửi dữ liệu quá N giây $\Rightarrow$ sensor_offline.

REST API

Dùng FastAPI, tối thiểu các endpoint:

GET /health	GET /beds
GET /beds/{bed_id}/state	GET /beds/{bed_id}/events
POST /beds/{bed_id}/command	# body: {"target": "nurse_call", "action": "off", "reason": "manual"}

API có thể publish command trực tiếp lên MQTT, hoặc đọc/ghi trạng thái qua file/Redis do nhóm thiết kế — phải giải thích lựa chọn trong báo cáo.

1.10 Tích hợp ThingsBoard (uplink + RPC)

1. Tạo một thiết bị kiểu *Gateway* trên ThingsBoard, lấy ACCESS_TOKEN đặt vào TB_GATEWAY_TOKEN.
2. Khi khởi động, gateway connect từng giường rồi đẩy telemetry:

```
// publish -> v1/gateway/telemetry
{ "bed-01": [{"ts": 1750000000000,
  "values": {"heart_rate": 122, "spo2": 88.0, "body_temp": 38.9}}],
  "bed-02": [{"ts": 1750000000000,
  "values": {"heart_rate": 80, "spo2": 98.0, "body_temp": 36.7}}] }
```

3. Subscribe v1/gateway/rpc; khi điều dưỡng bấm nút trên dashboard ThingsBoard, server gửi RPC, gateway chuyển thành command MQTT cục bộ:

```
// receive <- v1/gateway/rpc
{ "device": "bed-01", "data": {"id": 7, "method": "setNurseCall", "params": false} }
// reply -> v1/gateway/rpc : {"device": "bed-01", "id": 7, "data": {"success": true}}
```

4. Trên ThingsBoard: tạo **dashboard** sinh hiệu (biểu đồ HR/SpO₂ theo thời gian, gauge giá trị hiện tại, bảng alarm), nút điều khiển gửi RPC; cấu hình **Alarm** theo ngưỡng (vd spo2 < 92 → alarm *Critical*) bằng Rule Chain.

1.11 Yêu cầu lưu trữ InfluxDB

Lưu tối thiểu 3 loại dữ liệu. Gợi ý measurement / tag / field:

- vital_signs — tag: ward_id, bed_id; field: heart_rate, spo2, body_temp, respiration_rate, systolic_bp, diastolic_bp.
- ward_events — tag: bed_id, event_type, severity; field: value, threshold.
- actuator_status — tag: bed_id; field: nurse_call, light, oxygen.

1.12 Yêu cầu Grafana Dashboard

Tối thiểu các panel:

1. Nhịp tim theo thời gian cho từng giường.
2. SpO₂ theo thời gian cho từng giường.
3. Nhiệt độ cơ thể & huyết áp theo giường.
4. Trạng thái nurse_call/oxygen/light theo giường.
5. Số lượng event theo mức độ (info/warning/critical) theo thời gian.
6. Bảng event gần nhất: bed_id, event_type, severity, action_taken.

Dashboard phải thể hiện mối liên hệ: khi SpO₂ của bed-01 tụt dưới ngưỡng thì event spo2_low xuất hiện và oxygen chuyển sang on.

1.13 Docker và ảo hóa

Tuân thủ đầy đủ mục 4–5 của Khung yêu cầu chung. Mỗi service tự lập trình (virtual_sensor, virtual_actuator, iot_gateway, gateway_api) có Dockerfile riêng; dùng volume cho InfluxDB và Grafana; gateway đọc TB_HOST/TB_GATEWAY_TOKEN từ .env.

1.14 Câu hỏi bắt buộc trong báo cáo

1. Vì sao cần edge gateway tại khoa thay vì để mỗi sensor gửi thẳng lên cloud? (Phân tích độ trễ, mất kết nối, an toàn bệnh nhân.)
2. Nhóm thiết kế topic hierarchy và message format y tế như thế nào? Vì sao?
3. Rule engine gồm những luật nào? Phân mức độ severity ra sao?
4. Luồng xử lý từ telemetry → phát hiện → command diễn ra thế nào?
5. Gateway đóng vai trò ThingsBoard Gateway như thế nào (connect, telemetry, RPC)?
6. So sánh dashboard Grafana (edge) và ThingsBoard (cloud): mỗi nơi phù hợp việc gì?
7. Vì sao trong container không nên dùng localhost để gọi service khác?
8. Nếu triển khai lên gateway phần cứng thật tại bệnh viện, cần thay đổi gì?

1.15 Phân công gợi ý cho nhóm 3 sinh viên

- **SV1:** virtual sensor, virtual actuator, thiết kế topic/message.
- **SV2:** edge gateway, rule engine, ghi InfluxDB.
- **SV3:** tích hợp ThingsBoard (gateway API + dashboard + alarm), REST API, Docker Compose, Grafana, README. Cả nhóm cùng tích hợp và debug.

1.16 Tiêu chí chấm điểm

Nội dung	Điểm
Kiến trúc Virtual Healthcare Gateway hợp lý	1.0
Thiết kế MQTT topic & message format rõ ràng, mở rộng được	1.0
Virtual sensor sinh hiệu hợp lý, có tình huống nguy cấp	1.0
Virtual actuator nhận command và publish status đúng	0.8
Gateway validate, normalize, lưu trạng thái giường chính xác	1.0
Rule engine phát hiện nguy cấp (có severity) và điều khiển tự động	1.2
Tích hợp ThingsBoard: telemetry gateway + RPC điều khiển từ xa	1.2
Ghi telemetry/event/status vào InfluxDB	0.8
Grafana & ThingsBoard dashboard thể hiện rõ dữ liệu, event, trạng thái	0.8
Docker Compose, Dockerfile, network, volume, env đúng	1.0
README, báo cáo, demo và giải thích hệ thống	0.7 + 0.5
Tổng	10.0

1.17 Yêu cầu nâng cao (cộng điểm khuyến khích)

1. Cấu hình username/password cho Mosquitto.
2. Shared Attributes trên ThingsBoard để chỉnh ngưỡng từ xa.
3. Health check cho các service trong Compose.
4. Phát hiện sensor_offline và actuator_no_response.
5. Endpoint API đổi ngưỡng rule engine khi đang chạy.
6. Alarm rule + thông báo (email/Telegram) trên ThingsBoard.
7. Unit test cho rule engine.
8. Mô phỏng mất kết nối MQTT và cơ chế reconnect.

1.18 Kết luận

Đề tài rèn cho sinh viên cách xây dựng một hệ thống IoT y tế ảo hóa có tính thực tế cao: vừa xử lý cục bộ tại biên (đảm bảo phản ứng tức thời với tình huống nguy cấp), vừa đồng bộ với nền tảng đám mây ThingsBoard để giám sát và điều khiển từ xa. Qua đó sinh viên vận dụng đồng thời MQTT, rule engine, InfluxDB, Grafana, ThingsBoard Gateway API, RPC, REST API và Docker Compose — đúng tinh thần lập trình và ảo hóa cho IoT.

Đề tài 2

Virtual Smart Parking & Traffic Gateway

2

2 Đề tài 2 — Virtual Smart Parking & Traffic Gateway

2.1 Tên đề tài

Xây dựng hệ thống **Virtual IoT Gateway cho bãi đỗ xe thông minh và điều khiển giao thông cục bộ**, thu thập tình trạng chỗ đỗ theo từng khu vực, tự động điều khiển barie, biển báo hướng dẫn và đèn tín hiệu qua MQTT, đồng thời đồng bộ hai chiều với ThingsBoard Cloud (telemetry + RPC).

2.2 Bối cảnh

Tại các bãi đỗ xe lớn và khu đô thị, việc tìm chỗ trống gây ùn tắc và lãng phí thời gian. Hệ thống đỗ xe thông minh dùng cảm biến để biết số chỗ trống theo thời gian thực, hiển thị lên biển hướng dẫn và tự động đóng/mở barie lối vào. Một edge gateway tại bãi xe có thể tổng hợp dữ liệu nhiều khu vực, quyết định cục bộ (đóng lối vào khi hết chỗ, điều phối hàng đợi) ngay cả khi mất kết nối cloud, đồng thời gửi dữ liệu lên ThingsBoard để vận hành tập trung nhiều bãi.

Mini-project yêu cầu giả lập toàn bộ cảm biến chỗ đỗ, thiết bị chấp hành và gateway bằng Python trong container Docker, triển khai bằng Docker Compose.

2.3 Mục tiêu của mini-project

1. Lập trình nhiều virtual sensor mô phỏng tình trạng chỗ đỗ của các khu vực khác nhau.
2. Xây dựng edge gateway nhận telemetry MQTT từ nhiều khu vực đỗ xe.
3. Thiết kế topic hierarchy và message format cho bãi đỗ xe.
4. Chuẩn hóa dữ liệu (số chỗ trống, tỉ lệ lấp đầy, độ dài hàng đợi).
5. Phát hiện sự kiện (hết chỗ, sắp đầy, ùn tắc) bằng rule engine.
6. Tự động điều khiển barie, biển báo và đèn tín hiệu qua MQTT.
7. Lưu telemetry, event và trạng thái actuator vào InfluxDB.
8. Tích hợp ThingsBoard: đẩy telemetry nhiều khu vực và nhận RPC điều khiển từ xa.
9. Dashboard giám sát trên Grafana (edge) và ThingsBoard (cloud).
10. Cung cấp REST API truy vấn tình trạng bãi và điều khiển thủ công.
11. Container hóa và triển khai bằng Docker Compose.

2.4 Mô tả bài toán

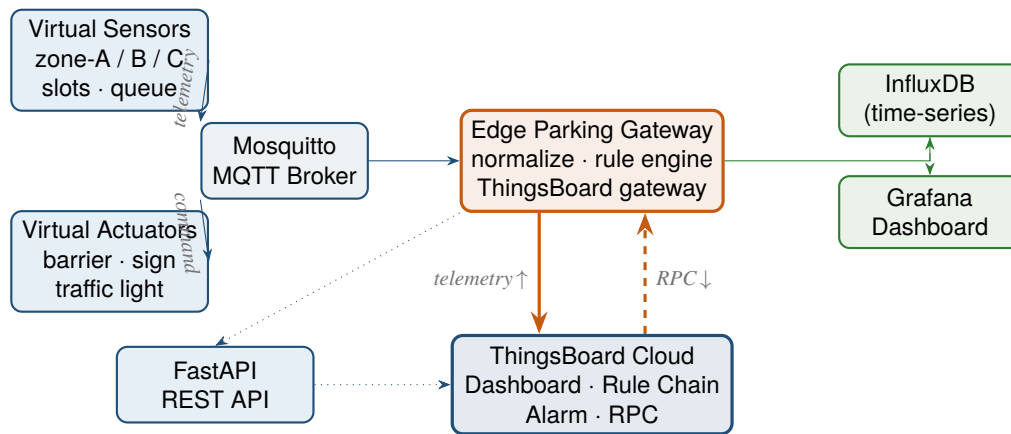
Hệ thống mô phỏng một bãi đỗ (lot-HUST) gồm tối thiểu 3 khu vực: zone-A, zone-B, zone-C. Mỗi khu vực có:

- Một **virtual sensor node** báo: tổng số chỗ, số chỗ đã chiếm, số chỗ trống, có xe chờ ở lối vào hay không, độ dài hàng đợi.
- Một **virtual actuator node**: barie lối vào (barrier), biển báo hướng dẫn (sign: hiển thị số chỗ trống hoặc “FULL”), đèn tín hiệu (light: xanh/đỏ cho lối vào).

Edge gateway nhận dữ liệu từ các khu vực, ra quyết định và điều khiển thiết bị khi hết chỗ hoặc ùn tắc.

2.5 Kiến trúc hệ thống yêu cầu

Kiến trúc tối thiểu như Hình 2.5.



Hình 2: Kiến trúc lai của Đề tài 2: gateway điều phối barie/biển báo cục bộ và đồng bộ với ThingsBoard Cloud.

2.6 Các service bắt buộc

- mosquitto — MQTT broker.
- sensor-zone-A/B/C — sensor mỗi khu vực.
- actuator-zone-A/B/C — actuator mỗi khu vực.
- parking-gateway — edge IoT gateway.
- influxdb — time-series database.
- grafana — dashboard edge.
- parking-api — REST API (FastAPI).

Phải đủ các nhóm chức năng: sensor simulator, actuator simulator, broker, gateway, database, dashboard, API.

2.7 Thiết kế MQTT topic

```

parking/zone-A/sensor/telemetry      # slot occupancy from zone sensor
parking/zone-A/actuator/command      # command to barrier / sign / light
parking/zone-A/actuator/status        # actuator status feedback
parking/zone-A/gateway/normalized     # normalized data
parking/zone-A/gateway/event          # event raised by gateway
  
```

2.8 Message format yêu cầu

Telemetry từ sensor:

```

{
  "device_id": "sensor-zone-A", "lot_id": "lot-HUST", "zone_id": "zone-A",
  "total_slots": 40, "occupied_slots": 33, "free_slots": 7,
  "entry_car_waiting": true, "queue_length": 3,
  "timestamp": "2026-06-10T10:00:00Z"
}
  
```

Command từ gateway tới actuator (target ∈ {barrier, sign, light}):

```

{ "zone_id": "zone-A", "target": "barrier", "action": "open",
  "reason": "slot_available", "timestamp": "2026-06-10T10:00:05Z" }
  
```

Status từ actuator:

```

{ "device_id": "actuator-zone-A", "zone_id": "zone-A",
  "barrier": "open", "sign": "7 FREE", "light": "green",
  "last_command_reason": "slot_available", "timestamp": "2026-06-10T10:00:06Z" }
  
```

Event từ gateway:

```
{ "zone_id": "zone-A", "event_type": "zone_full", "severity": "warning",
  "value": 0, "threshold": 0, "action_taken": "barrier_close,sign_FULL",
  "timestamp": "2026-06-10T10:00:05Z" }
```

2.9 Yêu cầu lập trình

Virtual sensor

1. Đọc env: LOT_ID, ZONE_ID, DEVICE_ID, MQTT_BROKER, TOTAL_SLOTS, PUBLISH_INTERVAL.
2. Mô phỏng xe vào/ra: occupied_slots thay đổi dần theo thời gian (tăng giờ cao điểm, giảm giờ thấp điểm), free_slots = total - occupied.
3. Sinh tình huống: bãi gần đầy, đầy hẵn, hàng đợi tăng ở lối vào.
4. Publish JSON lên parking/<zone>/sensor/telemetry.

Virtual actuator

1. Subscribe topic command của khu vực; parse JSON.
2. Cập nhật trạng thái barrier (open/close), sign (chuỗi hiển thị), light (green/red).
3. Publish lại trạng thái lên topic status; ghi log.

Edge IoT gateway

1. Subscribe telemetry tất cả khu vực; validate và normalize; tính occupancy_rate = occupied/total.
2. Lưu trạng thái mới nhất từng khu vực; ghi telemetry vào InfluxDB.
3. Chạy rule engine; sinh và publish event; gửi command tới actuator.
4. Đẩy telemetry lên ThingsBoard và nhận RPC (mục 2.10).

Rule engine

Tối thiểu 4 luật:

1. free_slots == 0 $\Rightarrow$ sign="FULL", barrier=close, light=red; event zone_full, severity warning.
2. free_slots > 0 và entry_car_waiting=true $\Rightarrow$ barrier=open, light=green; event admit_vehicle, severity info.
3. occupancy_rate >= 0.9 (chưa đầy hẵn) $\Rightarrow$ sign="ALMOST FULL"; event zone_almost_full, severity info.
4. queue_length > 5 $\Rightarrow$ event congestion, severity warning (gợi ý điều hướng sang khu vực còn trống).
5. (Mở rộng) barie mở quá N giây mà không có xe vào $\Rightarrow$ barrier=close; sensor offline $\Rightarrow$ sensor_offline.

REST API

GET /health	GET /zones
GET /zones/{zone_id}/state	GET /zones/{zone_id}/events
POST /zones/{zone_id}/command	# body: {"target": "barrier", "action": "open", "reason": "manual"}

2.10 Tích hợp ThingsBoard (uplink + RPC)

1. Tạo thiết bị Gateway, đặt token vào TB_GATEWAY_TOKEN; connect mỗi khu vực rồi đẩy telemetry:

```
// publish -> v1/gateway/telemetry
{ "zone-A": [{"ts": 1750000000000, "values": {"free_slots": 0, "occupancy_rate": 1.0, "queue_length": 6}}],
  "zone-B": [{"ts": 1750000000000, "values": {"free_slots": 12, "occupancy_rate": 0.7, "queue_length": 0}}]
}
```

2. Nhận RPC để điều khiển thủ công barie/biển báo từ trung tâm điều hành:

```
// receive <- v1/gateway/rpc
{ "device": "zone-A", "data": {"id": 3, "method": "setBarrier", "params": "open"} }
// reply -> v1/gateway/rpc : { "device": "zone-A", "id": 3, "data": {"success": true} }
```

3. Trên ThingsBoard: dashboard hiển thị tỉ lệ lấp đầy từng khu vực (gauge), số chỗ trống, bản đồ bãi; nút RPC setBarrier/setSign/setLight; Alarm khi một khu vực zone_full kéo dài quá ngưỡng thời gian.

2.11 Yêu cầu lưu trữ InfluxDB

- parking_occupancy — tag: lot_id, zone_id; field: total_slots, occupied_slots, free_slots, occupancy_rate, queue_length.
- gateway_events — tag: zone_id, event_type, severity; field: value, threshold.
- actuator_status — tag: zone_id; field: barrier, light (mã hóa số), sign (chuỗi).

2.12 Yêu cầu Grafana Dashboard

1. Tỉ lệ lấp đầy theo thời gian cho từng khu vực.
2. Số chỗ trống hiện tại (stat/gauge) theo khu vực.
3. Độ dài hàng đợi theo thời gian.
4. Trạng thái barie/biển báo/đèn theo khu vực.
5. Số event (zone_full, congestion) theo thời gian.
6. Bảng event gần nhất: zone_id, event_type, severity, action_taken.

2.13 Docker và ảo hóa

Tuân thủ mục 4–5 Khung yêu cầu chung: mỗi service tự lập trình có Dockerfile riêng; volume cho InfluxDB/Grafana; cấu hình qua .env; service gọi nhau bằng tên.

2.14 Câu hỏi bắt buộc trong báo cáo

1. Vì sao cần edge gateway cho bãi xe thay vì để mỗi cảm biến gửi thẳng lên cloud?
2. Nhóm thiết kế topic và message format thế nào cho nhiều khu vực?
3. Rule engine điều phối barie/biển báo theo những luật nào?
4. Khi một khu vực hết chỗ, luồng xử lý từ telemetry → command diễn ra ra sao?
5. Gateway đóng vai trò ThingsBoard Gateway như thế nào?
6. Khi nào nên quyết định cục bộ tại edge, khi nào cần lên cloud?
7. Vì sao container không nên dùng localhost để gọi service khác?
8. Muốn mở rộng quản lý nhiều bãi xe trên ThingsBoard cần thay đổi gì?

2.15 Phân công gợi ý cho nhóm 3 sinh viên

- **SV1:** virtual sensor, virtual actuator, thiết kế topic/message.
- **SV2:** edge gateway, rule engine, ghi InfluxDB.
- **SV3:** tích hợp ThingsBoard + dashboard + alarm, REST API, Docker Compose, Grafana, README.

2.16 Tiêu chí chấm điểm

Nội dung	Điểm
Kiến trúc Virtual Parking Gateway hợp lý	1.0
Thiết kế MQTT topic & message format rõ ràng, mở rộng được	1.0
Virtual sensor mô phỏng vào/ra và tình huống đầy bãi hợp lý	1.0
Virtual actuator (barie/biển báo/đèn) nhận command và publish status	0.8
Gateway validate, normalize, tính tỉ lệ lấp đầy, lưu trạng thái	1.0
Rule engine điều phối barie/biển báo và xử lý ùn tắc tự động	1.2
Tích hợp ThingsBoard: telemetry gateway + RPC điều khiển từ xa	1.2
Ghi telemetry/event/status vào InfluxDB	0.8
Grafana & ThingsBoard dashboard thể hiện rõ tình trạng bãi	0.8
Docker Compose, Dockerfile, network, volume, env đúng	1.0
README, báo cáo, demo và giải thích hệ thống	0.7 + 0.5
Tổng	10.0

2.17 Yêu cầu nâng cao (cộng điểm khuyến khích)

- Bản đồ chỗ đỗ (slot map) trực quan trên ThingsBoard.
- Gợi ý điều hướng sang khu vực còn trống khi một khu vực đầy.
- Thống kê thời lượng đỗ trung bình theo khu vực.
- Shared Attributes để đổi ngưỡng queue_length từ xa.
- Health check cho các service trong Compose.
- Mô phỏng nhiều lối vào/ra và điều phối đèn tín hiệu.
- Unit test cho rule engine.
- Cơ chế reconnect khi mất kết nối MQTT/ThingsBoard.

2.18 Kết luận

Đề tài giúp sinh viên xây dựng hệ thống đỗ xe/giao thông ảo hóa với khả năng ra quyết định cục bộ tại biên và vận hành tập trung trên cloud. Sinh viên thực hành trọn vẹn chuỗi công nghệ MQTT, rule engine, InfluxDB, Grafana, ThingsBoard Gateway API, RPC, REST API và Docker Compose trong một bài toán đô thị thông minh thực tế.

Đề tài 3

Virtual Smart Water & Flood Early-Warning Gateway **3**

3 Đề tài 3 — Virtual Smart Water & Flood Early-Warning Gateway

3.1 Tên đề tài

Xây dựng hệ thống **Virtual IoT Gateway giám sát nguồn nước và cảnh báo sớm ngập lụt**, thu thập mực nước, lưu lượng, lượng mưa và chất lượng nước tại nhiều trạm, tự động điều khiển bơm tiêu úng, cửa xả và còi cảnh báo qua MQTT, đồng bộ hai chiều với ThingsBoard Cloud (telemetry + RPC).

3.2 Bối cảnh

Ngập lụt đô thị và lũ trên sông gây thiệt hại lớn; cảnh báo sớm vài phút cũng có giá trị. Hệ thống quan trắc đặt cảm biến tại nhiều trạm dọc sông/kênh tiêu để theo dõi mực nước, lưu lượng và lượng mưa. Một edge gateway tại trạm có thể phát hiện mực nước vượt ngưỡng và lập tức kích hoạt bơm, mở cửa xả, hú còi mà không phụ thuộc đường truyền lên trung tâm; đồng thời đẩy dữ liệu lên ThingsBoard để trung tâm điều hành theo dõi toàn lưu vực và ra lệnh từ xa.

Mini-project yêu cầu giả lập toàn bộ cảm biến, thiết bị chấp hành và gateway bằng Python trong container Docker, triển khai bằng Docker Compose.

3.3 Mục tiêu của mini-project

1. Lập trình nhiều virtual sensor mô phỏng các trạm quan trắc khác nhau.
2. Xây dựng edge gateway nhận telemetry MQTT từ nhiều trạm.
3. Thiết kế topic hierarchy và message format cho quan trắc nước.
4. Chuẩn hóa dữ liệu (mực nước, lưu lượng, lượng mưa, chất lượng nước).
5. Phát hiện cảnh báo nhiều mức (advisory/warning/emergency) bằng rule engine.
6. Tự động điều khiển bơm, cửa xả, còi và bảng cảnh báo qua MQTT.
7. Lưu telemetry, event và trạng thái actuator vào InfluxDB.
8. Tích hợp ThingsBoard: đẩy telemetry nhiều trạm và nhận RPC điều khiển từ xa.
9. Dashboard giám sát trên Grafana (edge) và ThingsBoard (cloud).
10. Cung cấp REST API truy vấn trạng thái trạm và điều khiển thủ công.
11. Container hóa và triển khai bằng Docker Compose.

3.4 Mô tả bài toán

Hệ thống mô phỏng một lưu vực gồm tối thiểu 3 trạm: station-01, station-02, station-03. Mỗi trạm có:

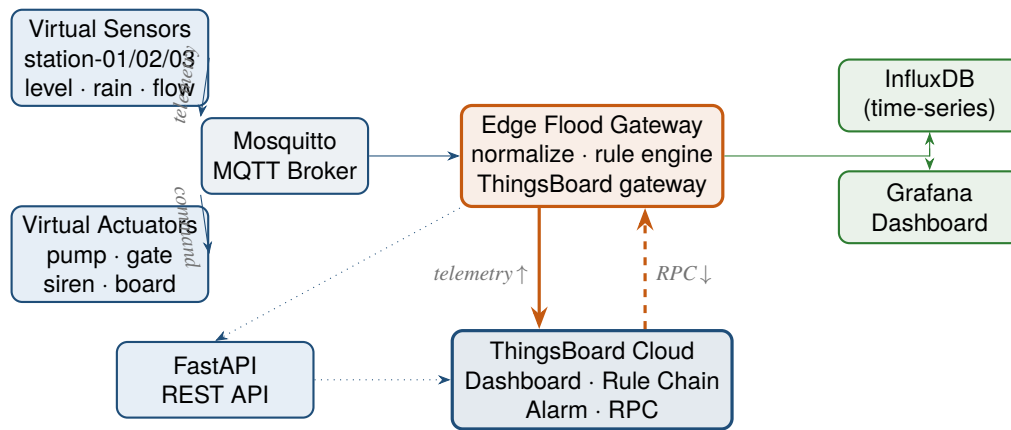
- Một **virtual sensor node** gửi định kỳ: mực nước (m), lưu lượng (m^3/s), lượng mưa (mm/h), độ đục (NTU), pH.
- Một **virtual actuator node**: bơm tiêu úng (pump), cửa xả/van (gate), còi cảnh báo (siren), bảng cảnh báo (board: hiển thị advisory/warning/emergency).

Edge gateway nhận dữ liệu, phát hiện nguy cơ ngập và điều khiển thiết bị tương ứng.

3.5 Kiến trúc hệ thống yêu cầu

Kiến trúc tối thiểu như Hình 3.5.

3.6 Các service bắt buộc



Hình 3: Kiến trúc lại của Đề tài 3: gateway cảnh báo và điều khiển tiêu ứng cục bộ, đồng bộ với ThingsBoard Cloud.

- mosquitto — MQTT broker.
- sensor-station-01/02/03 — sensor mỗi trạm.
- actuator-station-01/02/03 — actuator mỗi trạm.
- flood-gateway — edge IoT gateway.
- influxdb — time-series database.
- grafana — dashboard edge.
- flood-api — REST API (FastAPI).

Phải đủ các nhóm chức năng: sensor simulator, actuator simulator, broker, gateway, database, dashboard, API.

3.7 Thiết kế MQTT topic

```

basin/station-01/sensor/telemetry      # raw measurements from station
basin/station-01/actuator/command      # command to pump / gate / siren / board
basin/station-01/actuator/status       # actuator status feedback
basin/station-01/gateway/normalized    # normalized data
basin/station-01/gateway/event         # alert raised by gateway
  
```

3.8 Message format yêu cầu

Telemetry từ sensor:

```

{
  "device_id": "sensor-station-01", "basin_id": "to-lich", "station_id": "station-01",
  "water_level": 3.42, "flow_rate": 12.7, "rainfall": 45.0,
  "turbidity": 120.0, "ph": 7.1,
  "timestamp": "2026-06-10T10:00:00Z"
}
  
```

Command từ gateway tới actuator (target ∈ {pump, gate, siren, board}):

```

{ "station_id": "station-01", "target": "pump", "action": "on",
  "reason": "water_level_high", "timestamp": "2026-06-10T10:00:05Z" }
  
```

Status từ actuator:

```

{ "device_id": "actuator-station-01", "station_id": "station-01",
  "pump": "on", "gate": "open", "siren": "off", "board": "warning",
  "last_command_reason": "water_level_high", "timestamp": "2026-06-10T10:00:06Z" }
  
```

Event từ gateway:

```

{ "station_id": "station-01", "event_type": "flood_warning", "severity": "warning",
  "value": 3.42, "threshold": 3.00, "action_taken": "pump_on,gate_open,board_warning",
  "timestamp": "2026-06-10T10:00:07Z" }
  
```

```
"timestamp": "2026-06-10T10:00:05Z" }
```

3.9 Yêu cầu lập trình

Virtual sensor

1. Đọc env: BASIN_ID, STATION_ID, DEVICE_ID, MQTT_BROKER, PUBLISH_INTERVAL.
2. Mô phỏng mực nước biến đổi theo lượng mưa (mưa lớn → mực nước dâng dần), lưu giá trị trước và thay đổi có xu hướng, không random độc lập.
3. Sinh tình huống: mưa to kéo dài, mực nước vượt ngưỡng, nước đục/pH bất thường.
4. Publish JSON lên basin/<station>/sensor/telemetry.

Virtual actuator

1. Subscribe topic command của trạm; parse JSON.
2. Cập nhật trạng thái pump, gate, siren, board.
3. Publish lại trạng thái lên topic status; ghi log.

Edge IoT gateway

1. Subscribe telemetry tất cả trạm; validate và normalize; tính tốc độ dâng mực nước (so với giá trị trước).
2. Lưu trạng thái mới nhất từng trạm; ghi telemetry vào InfluxDB.
3. Chạy rule engine; sinh và publish event; gửi command tới actuator.
4. Đẩy telemetry lên ThingsBoard và nhận RPC (mục 3.10).

Rule engine

Tối thiểu 4 luật (phân mức advisory/warning/emergency):

1. $\text{water_level} > 3.0 \Rightarrow \text{pump}=\text{on}, \text{gate}=\text{open}, \text{board}=\text{warning}; \text{event flood_warning}, \text{severity warning}.$
2. $\text{water_level} > 4.0 \Rightarrow \text{thêm siren}=\text{on}, \text{board}=\text{emergency}; \text{event flood_emergency}, \text{severity emergency}.$
3. $\text{rainfall} > 40$ và mực nước đang dâng $\Rightarrow \text{board}=\text{advisory}; \text{event heavy_rain}, \text{severity advisory}.$
4. $\text{turbidity} > 100$ hoặc $\text{ph} < 6 / \text{ph} > 9 \Rightarrow \text{event water_quality_alert}, \text{severity warning}.$
5. (Mở rộng) thời gian chạy bơm vượt ngưỡng $\Rightarrow \text{pump_maintenance}; \text{sensor offline} \Rightarrow \text{sensor_offline}.$

REST API

```
GET /health
GET /stations/{station_id}/state
POST /stations/{station_id}/command

GET /stations
GET /stations/{station_id}/events
# body: {"target":"pump","action":"off","reason":"manual"}
```

3.10 Tích hợp ThingsBoard (uplink + RPC)

1. Tạo thiết bị Gateway, đặt token vào TB_GATEWAY_TOKEN; connect mỗi trạm rồi đẩy telemetry:

```
// publish -> v1/gateway/telemetry
{ "station-01": [{"ts":1750000000000,"values":{"water_level":4.2,"rainfall":60,"flow_rate":18.3}}],
  "station-02": [{"ts":1750000000000,"values":{"water_level":2.1,"rainfall":15,"flow_rate":7.0}}] }
```

2. Nhận RPC để điều khiển bơm/cửa xả/còi từ trung tâm điều hành:

```
// receive <- v1/gateway/rpc
{ "device":"station-01", "data":{"id":5,"method":"setPump","params":true} }
// reply -> v1/gateway/rpc : {"device":"station-01","id":5,"data":{"success":true}}
```

3. Trên ThingsBoard: dashboard mức nước theo thời gian, bản đồ trạm, mức cảnh báo; nút RPC setPump/setGate/setSiren; Alarm nhiều mức theo ngưỡng mức nước (warning/emergency) cấu hình bằng Rule Chain.

3.11 Yêu cầu lưu trữ InfluxDB

- water_telemetry — tag: basin_id, station_id; field: water_level, flow_rate, rainfall, turbidity, ph.
- gateway_events — tag: station_id, event_type, severity; field: value, threshold.
- actuator_status — tag: station_id; field: pump, gate, siren, board.

3.12 Yêu cầu Grafana Dashboard

1. Mức nước theo thời gian cho từng trạm (kèm đường ngưỡng cảnh báo).
2. Lượng mưa theo thời gian cho từng trạm.
3. Lưu lượng và chất lượng nước (turbidity, pH).
4. Trạng thái bơm/cửa xả/còi/bảng cảnh báo theo trạm.
5. Số event theo mức (advisory/warning/emergency) theo thời gian.
6. Bảng event gần nhất: station_id, event_type, severity, action_taken.

3.13 Docker và ảo hóa

Tuân thủ mục 4–5 Khung yêu cầu chung: mỗi service tự lập trình có Dockerfile riêng; volume cho InfluxDB/Grafana; cấu hình qua .env; service gọi nhau bằng tên.

3.14 Câu hỏi bắt buộc trong báo cáo

1. Vì sao cảnh báo lũ cần xử lý tại edge thay vì chờ xử lý trên cloud?
2. Nhóm thiết kế topic và message format cho quan trắc nước thế nào?
3. Rule engine phân mức cảnh báo (advisory/warning/emergency) ra sao?
4. Khi mức nước vượt ngưỡng, luồng telemetry → cảnh báo → điều khiển diễn ra thế nào?
5. Gateway đóng vai trò ThingsBoard Gateway như thế nào?
6. Vì sao đẩy dữ liệu lên cloud vẫn cần, dù đã xử lý tại edge?
7. Vì sao container không nên dùng localhost để gọi service khác?
8. Muốn mở rộng giám sát toàn thành phố trên ThingsBoard cần thay đổi gì?

3.15 Phân công gợi ý cho nhóm 3 sinh viên

- **SV1:** virtual sensor, virtual actuator, thiết kế topic/message.
- **SV2:** edge gateway, rule engine, ghi InfluxDB.
- **SV3:** tích hợp ThingsBoard + dashboard + alarm, REST API, Docker Compose, Grafana, README.

3.16 Tiêu chí chấm điểm

Nội dung	Điểm
Kiến trúc Virtual Flood Gateway hợp lý	1.0
Thiết kế MQTT topic & message format rõ ràng, mở rộng được	1.0
Virtual sensor mô phỏng mực nước/mưa có xu hướng và tình huống lũ	1.0
Virtual actuator (bơm/cửa xả/còi/bảng) nhận command và publish status	0.8
Gateway validate, normalize, tính tốc độ dâng, lưu trạng thái	1.0
Rule engine cảnh báo nhiều mức và điều khiển tiêu ứng tự động	1.2
Tích hợp ThingsBoard: telemetry gateway + RPC điều khiển từ xa	1.2
Ghi telemetry/event/status vào InfluxDB	0.8
Grafana & ThingsBoard dashboard thể hiện rõ mực nước, cảnh báo, trạng thái	0.8
Docker Compose, Dockerfile, network, volume, env đúng	1.0
README, báo cáo, demo và giải thích hệ thống	0.7 + 0.5
Tổng	10.0

3.17 Yêu cầu nâng cao (cộng điểm khuyến khích)

1. Dự báo xu hướng mực nước ngắn hạn từ tốc độ dâng.
2. Bản đồ trạm và mức cảnh báo theo màu trên ThingsBoard.
3. Shared Attributes để chỉnh ngưỡng cảnh báo từ xa.
4. Health check cho các service trong Compose.
5. Alarm + thông báo (email/Telegram) khi vào mức emergency.
6. Phát hiện `sensor_offline` và mất kết nối trạm.
7. Unit test cho rule engine.
8. Cơ chế reconnect khi mất kết nối MQTT/ThingsBoard.

3.18 Kết luận

Đề tài giúp sinh viên xây dựng hệ thống quan trắc nước và cảnh báo lũ ảo hóa, nhấn mạnh vai trò xử lý thời gian thực tại biên cho các tình huống khẩn cấp, kết hợp với giám sát toàn lưu vực trên ThingsBoard Cloud. Sinh viên vận dụng MQTT, rule engine nhiều mức, InfluxDB, Grafana, ThingsBoard Gateway API, RPC, REST API và Docker Compose.

Đề tài 4

Virtual Smart Air Quality Gateway

4

4 Đề tài 4 — Virtual Smart Air Quality Gateway

4.1 Tên đề tài

Xây dựng hệ thống **Virtual IoT Gateway giám sát chất lượng không khí đô thị**, thu thập bụi mịn và khí ô nhiễm tại nhiều trạm, tính chỉ số AQI, tự động điều khiển quạt thông gió, máy lọc khí và bảng cảnh báo qua MQTT, đồng bộ hai chiều với ThingsBoard Cloud (telemetry + RPC).

4.2 Bối cảnh

Ô nhiễm không khí đô thị (bụi mịn PM2.5, PM10, CO, NO₂, O₃) ảnh hưởng trực tiếp tới sức khỏe. Mạng lưới trạm quan trắc đặt tại nhiều khu vực giúp theo dõi chất lượng không khí theo thời gian thực. Một edge gateway tại trạm có thể tính chỉ số AQI cục bộ, bật máy lọc khí/quạt thông gió và hiển thị cảnh báo cho người dân ngay khi chất lượng xấu, đồng thời đẩy dữ liệu lên ThingsBoard để xây bản đồ ô nhiễm toàn thành phố và điều khiển từ xa.

Mini-project yêu cầu giả lập toàn bộ cảm biến, thiết bị chấp hành và gateway bằng Python trong container Docker, triển khai bằng Docker Compose.

4.3 Mục tiêu của mini-project

1. Lập trình nhiều virtual sensor mô phỏng các trạm quan trắc không khí khác nhau.
2. Xây dựng edge gateway nhận telemetry MQTT từ nhiều trạm.
3. Thiết kế topic hierarchy và message format cho quan trắc không khí.
4. Chuẩn hóa dữ liệu và **tính chỉ số AQI** tại gateway.
5. Phát hiện mức ô nhiễm và cảnh báo theo cấp AQI bằng rule engine.
6. Tự động điều khiển quạt, máy lọc khí, phun sương và bảng cảnh báo qua MQTT.
7. Lưu telemetry, AQI, event và trạng thái actuator vào InfluxDB.
8. Tích hợp ThingsBoard: đẩy telemetry nhiều trạm và nhận RPC điều khiển từ xa.
9. Dashboard giám sát trên Grafana (edge) và ThingsBoard (cloud).
10. Cung cấp REST API truy vấn AQI từng trạm và điều khiển thủ công.
11. Container hóa và triển khai bằng Docker Compose.

4.4 Mô tả bài toán

Hệ thống mô phỏng mạng quan trắc gồm tối thiểu 3 trạm: station-01, station-02, station-03. Mỗi trạm có:

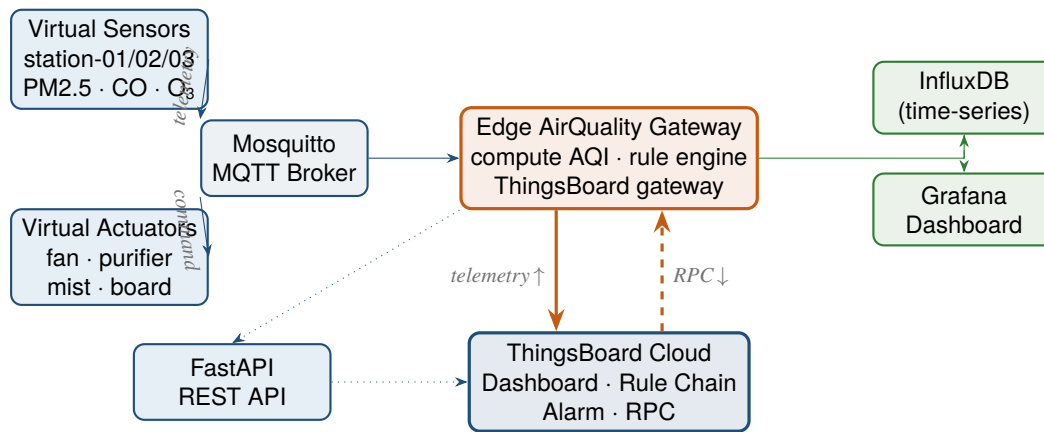
- Một **virtual sensor node** gửi định kỳ: PM2.5, PM10 ($\mu\text{g}/\text{m}^3$), CO (ppm), NO₂, O₃ (ppb), nhiệt độ, độ ẩm.
- Một **virtual actuator node**: quạt thông gió (fan), máy lọc khí (purifier), hệ phun sương giảm bụi (mist), bảng cảnh báo (board: Good/Moderate/Unhealthy).

Edge gateway nhận dữ liệu, tính AQI, phát hiện ô nhiễm và điều khiển thiết bị.

4.5 Kiến trúc hệ thống yêu cầu

Kiến trúc tối thiểu như Hình 4.5.

4.6 Các service bắt buộc



Hình 4: Kiến trúc lại của Đề tài 4: gateway tính AQI và điều khiển lọc khí cục bộ, đồng bộ với ThingsBoard Cloud.

- mosquitto — MQTT broker.
- sensor-station-01/02/03 — sensor mỗi trạm.
- actuator-station-01/02/03 — actuator mỗi trạm.
- air-gateway — edge IoT gateway.
- influxdb — time-series database.
- grafana — dashboard edge.
- air-api — REST API (FastAPI).

Phải đủ các nhóm chức năng: sensor simulator, actuator simulator, broker, gateway, database, dashboard, API.

4.7 Thiết kế MQTT topic

```

city/station-01/sensor/telemetry      # raw air measurements from station
city/station-01/actuator/command      # command to fan / purifier / mist / board
city/station-01/actuator/status       # actuator status feedback
city/station-01/gateway/normalized    # normalized data + computed AQI
city/station-01/gateway/event         # event raised by gateway
  
```

4.8 Message format yêu cầu

Telemetry từ sensor:

```

{
  "device_id": "sensor-station-01", "city_id": "hanoi", "station_id": "station-01",
  "pm2_5": 165.0, "pm10": 210.0, "co_ppm": 4.5,
  "no2_ppb": 80.0, "o3_ppb": 60.0, "temperature": 31.2, "humidity": 70.0,
  "timestamp": "2026-06-10T10:00:00Z"
}
  
```

Command từ gateway tới actuator (target ∈ {fan, purifier, mist, board}):

```

{ "station_id": "station-01", "target": "purifier", "action": "on",
  "reason": "aqi_unhealthy", "timestamp": "2026-06-10T10:00:05Z" }
  
```

Status từ actuator:

```

{ "device_id": "actuator-station-01", "station_id": "station-01",
  "fan": "on", "purifier": "on", "mist": "off", "board": "Unhealthy",
  "last_command_reason": "aqi_unhealthy", "timestamp": "2026-06-10T10:00:06Z" }
  
```

Event từ gateway (kèm AQI đã tính):

```

{ "station_id": "station-01", "event_type": "aqi_unhealthy", "severity": "warning",
  "aqi": 168, "dominant": "pm2_5", "value": 165.0, "threshold": 150,
  "timestamp": "2026-06-10T10:00:07Z" }
  
```

```
"action_taken": "purifier_on,fan_on,board_Unhealthy",
"timestamp": "2026-06-10T10:00:05Z" }
```

4.9 Yêu cầu lập trình

Virtual sensor

1. Đọc env: CITY_ID, STATION_ID, DEVICE_ID, MQTT_BROKER, PUBLISH_INTERVAL.
2. Mô phỏng nồng độ bụi/khí biến đổi theo thời gian (cao điểm giao thông sáng/chiều), có xu hướng, không random độc lập.
3. Sinh tình huống: bụi mịn tăng đột biến, CO vượt ngưỡng.
4. Publish JSON lên city/<station>/sensor/telemetry.

Virtual actuator

1. Subscribe topic command của trạm; parse JSON.
2. Cập nhật trạng thái fan, purifier, mist, board.
3. Publish lại trạng thái lên topic status; ghi log.

Edge IoT gateway

1. Subscribe telemetry tất cả trạm; validate và normalize.
2. **Tính AQI** từ PM2.5/PM10 (theo bảng breakpoint của US EPA, có thể đơn giản hóa) và xác định chất ô nhiễm *dominant*.
3. Lưu trạng thái mới nhất từng trạm; ghi telemetry + AQI vào InfluxDB.
4. Chạy rule engine; sinh và publish event; gửi command tới actuator.
5. Đẩy telemetry (kèm AQI) lên ThingsBoard và nhận RPC (mục 4.10).

Gợi ý tính AQI

AQI cho mỗi chất được nội suy tuyến tính theo các khoảng breakpoint ($I = \frac{I_{hi}-I_{lo}}{C_{hi}-C_{lo}}(C - C_{lo}) + I_{lo}$).
 AQI tổng của trạm là *giá trị lớn nhất* trong các AQI thành phần; chất cho giá trị lớn nhất là *dominant*.
 Cấp: 0–50 Good, 51–100 Moderate, 101–150 USG, 151–200 Unhealthy, >200 Very Unhealthy.

Rule engine

Tối thiểu 4 luật (theo cấp AQI):

1. aqi > 150 (Unhealthy) $\Rightarrow$ purifier=on, fan=on, board="Unhealthy"; event aqi_unhealthy, severity warning.
2. aqi > 200 $\Rightarrow$ thêm mist=on, board="Very Unhealthy"; severity critical.
3. co_ppm > 9 $\Rightarrow$ fan=on mức tối đa; event co_high, severity critical (CO nguy hiểm trực tiếp).
4. aqi <= 50 (Good) $\Rightarrow$ tắt fan/purifier/mist, board="Good"; event aqi_good, severity info.
5. (Mở rộng) giá trị sensor trôi/đứng yên bất thường $\Rightarrow$ sensor_fault; sensor offline $\Rightarrow$ sensor_offline.

REST API

GET /health	GET /stations
GET /stations/{station_id}/state	GET /stations/{station_id}/events
POST /stations/{station_id}/command	# body: {"target":"purifier","action":"off","reason":"manual"}

4.10 Tích hợp ThingsBoard (uplink + RPC)

1. Tạo thiết bị Gateway, đặt token vào TB_GATEWAY_TOKEN; connect mỗi trạm rồi đẩy telemetry kèm AQI:

```
// publish -> v1/gateway/telemetry
{ "station-01": [{"ts":1750000000000,"values":{"pm2_5":165,"aqi":168,"co_ppm":4.5}}],
  "station-02": [{"ts":1750000000000,"values":{"pm2_5":40,"aqi":78,"co_ppm":1.0}}] }
```

2. Nhận RPC để điều khiển máy lọc/quạt từ trung tâm:

```
// receive <- v1/gateway/rpc
{ "device":"station-01", "data":{"id":9,"method":"setPurifier","params":true} }
// reply -> v1/gateway/rpc : {"device":"station-01","id":9,"data":{"success":true}}
```

3. Trên ThingsBoard: dashboard bản đồ AQI theo màu (Good–Hazardous), biểu đồ PM2.5/PM10 theo thời gian, gauge AQI; nút RPC setFan/setPurifier; Alarm theo cấp AQI cấu hình bằng Rule Chain.

4.11 Yêu cầu lưu trữ InfluxDB

- air_quality — tag: city_id, station_id; field: pm2_5, pm10, co_ppm, no2_ppb, o3_ppb, aqi, temperature, humidity.
- gateway_events — tag: station_id, event_type, severity, dominant; field: aqi, value, threshold.
- actuator_status — tag: station_id; field: fan, purifier, mist, board.

4.12 Yêu cầu Grafana Dashboard

1. AQI theo thời gian cho từng trạm (tô màu theo cấp).
2. PM2.5 và PM10 theo thời gian.
3. Nồng độ CO/NO₂/O₃ theo trạm.
4. Trạng thái quạt/máy lọc/phun sương/bảng cảnh báo theo trạm.
5. Số event theo cấp AQI theo thời gian.
6. Bảng event gần nhất: station_id, event_type, aqi, severity, action_taken.

4.13 Docker và ảo hóa

Tuân thủ mục 4–5 Khung yêu cầu chung: mỗi service tự lập trình có Dockerfile riêng; volume cho InfluxDB/Grafana; cấu hình qua .env; service gọi nhau bằng tên.

4.14 Câu hỏi bắt buộc trong báo cáo

1. Vì sao nên tính AQI tại gateway thay vì gửi dữ liệu thô lên cloud rồi mới tính?
2. Nhóm thiết kế topic và message format cho quan trắc không khí thế nào?
3. Cách tính AQI và xác định chất ô nhiễm dominant của nhóm?
4. Rule engine điều khiển lọc khí/quạt theo cấp AQI ra sao?
5. Gateway đóng vai trò ThingsBoard Gateway như thế nào?
6. So sánh dashboard Grafana (edge) và bản đồ AQI trên ThingsBoard (cloud).
7. Vì sao container không nên dùng localhost để gọi service khác?
8. Muốn mở rộng thành mạng quan trắc toàn thành phố cần thay đổi gì?

4.15 Phân công gợi ý cho nhóm 3 sinh viên

- **SV1:** virtual sensor, virtual actuator, thiết kế topic/message.
- **SV2:** edge gateway, tính AQI, rule engine, ghi InfluxDB.
- **SV3:** tích hợp ThingsBoard + dashboard + alarm, REST API, Docker Compose, Grafana, README.

4.16 Tiêu chí chấm điểm

Nội dung	Điểm
Kiến trúc Virtual Air Quality Gateway hợp lý	1.0
Thiết kế MQTT topic & message format rõ ràng, mở rộng được	1.0
Virtual sensor mô phỏng bụi/khí có xu hướng và tình huống ô nhiễm	1.0
Virtual actuator (quạt/lọc/sương/băng) nhận command và publish status	0.8
Gateway tính AQI chính xác, normalize, lưu trạng thái	1.2
Rule engine điều khiển lọc khí theo cấp AQI tự động	1.0
Tích hợp ThingsBoard: telemetry gateway + RPC điều khiển từ xa	1.2
Ghi telemetry/AQI/event/status vào InfluxDB	0.8
Grafana & ThingsBoard dashboard thể hiện rõ AQI, event, trạng thái	0.8
Docker Compose, Dockerfile, network, volume, env đúng	1.0
README, báo cáo, demo và giải thích hệ thống	0.7 + 0.5
Tổng	10.0

4.17 Yêu cầu nâng cao (cộng điểm khuyến khích)

1. Bản đồ nhiệt AQI (heatmap) toàn thành phố trên ThingsBoard.
2. Tính AQI đầy đủ cho nhiều chất (PM2.5, PM10, CO, O₃, NO₂).
3. Shared Attributes để chỉnh ngưỡng cảnh báo từ xa.
4. Health check cho các service trong Compose.
5. Alarm + thông báo khi AQI vào cấp Unhealthy.
6. Dự báo xu hướng AQI ngắn hạn.
7. Unit test cho hàm tính AQI và rule engine.
8. Cơ chế reconnect khi mất kết nối MQTT/ThingsBoard.

4.18 Kết luận

Đề tài giúp sinh viên xây dựng hệ thống quan trắc không khí ảo hóa có xử lý số liệu tại biên (tính AQI, điều khiển lọc khí) và giám sát tập trung trên ThingsBoard Cloud. Ngoài các kỹ thuật IoT cốt lõi, sinh viên thực hành thêm xử lý dữ liệu (tính chỉ số) tại gateway — một năng lực quan trọng của edge computing.

Đề tài 5

Virtual Smart Waste Management Gateway

5

5 Đề tài 5 — Virtual Smart Waste Management Gateway

5.1 Tên đề tài

Xây dựng hệ thống **Virtual IoT Gateway quản lý thùng rác thông minh và tối ưu thu gom**, theo dõi mức đầy, khối lượng, khí và nhiệt độ của nhiều thùng rác, tự động khóa nắp, nén rác, báo động và phát tín hiệu điều xe thu gom qua MQTT, đồng bộ hai chiều với ThingsBoard Cloud (telemetry + RPC).

5.2 Bối cảnh

Thu gom rác theo lịch cố định gây lãng phí (thùng chưa đầy vẫn đi thu) hoặc quá tải (thùng tràn gây mất vệ sinh). Thùng rác thông minh gắn cảm biến đo mức đầy, khối lượng, khí methane và nhiệt độ giúp thu gom *theo nhu cầu*. Một edge gateway khu vực tổng hợp trạng thái nhiều thùng, phát hiện thùng đầy/cháy/quá tải và sinh yêu cầu điều xe; đồng thời đẩy dữ liệu lên ThingsBoard để xây bản đồ thùng rác và tối ưu lộ trình thu gom toàn thành phố.

Mini-project yêu cầu giả lập toàn bộ cảm biến, thiết bị chấp hành và gateway bằng Python trong container Docker, triển khai bằng Docker Compose.

5.3 Mục tiêu của mini-project

1. Lập trình nhiều virtual sensor mô phỏng các thùng rác khác nhau.
2. Xây dựng edge gateway nhận telemetry MQTT từ nhiều thùng.
3. Thiết kế topic hierarchy và message format cho thùng rác thông minh.
4. Chuẩn hóa dữ liệu (mức đầy, khối lượng, khí, nhiệt độ).
5. Phát hiện sự kiện (đầy, cháy, quá tải, đổ nghiêng) bằng rule engine.
6. Tự động khóa nắp, nén rác, báo động và sinh yêu cầu thu gom qua MQTT.
7. Lưu telemetry, event và trạng thái actuator vào InfluxDB.
8. Tích hợp ThingsBoard: đẩy telemetry nhiều thùng và nhận RPC điều khiển từ xa.
9. Dashboard giám sát trên Grafana (edge) và ThingsBoard (cloud).
10. Cung cấp REST API truy vấn trạng thái thùng, danh sách cần thu gom và điều khiển thủ công.
11. Container hóa và triển khai bằng Docker Compose.

5.4 Mô tả bài toán

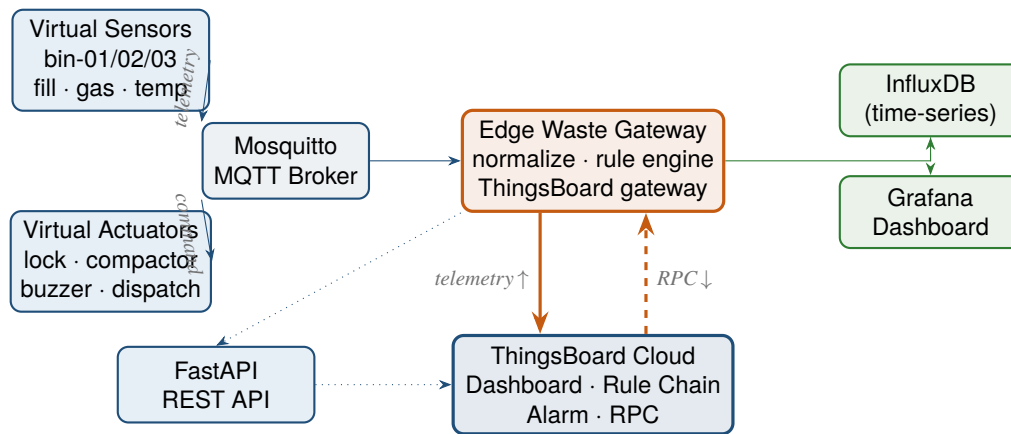
Hệ thống mô phỏng một khu vực gồm tối thiểu 3 thùng rác: bin-01, bin-02, bin-03. Mỗi thùng có:

- Một **virtual sensor node** gửi định kỳ: mức đầy (%), khối lượng (kg), nồng độ methane (ppm), nhiệt độ (°C), trạng thái nắp, cờ đổ nghiêng.
- Một **virtual actuator node**: khóa nắp (lock), bộ nén rác (compactor), còi báo động (buzzer), tín hiệu điều xe thu gom (dispatch).

Edge gateway nhận dữ liệu, phát hiện thùng cần xử lý và điều khiển thiết bị, đồng thời duy trì danh sách thùng cần thu gom.

5.5 Kiến trúc hệ thống yêu cầu

Kiến trúc tối thiểu như Hình 5.5.



Hình 5: Kiến trúc lai của Đề tài 5: gateway xử lý thùng rác cục bộ và đồng bộ với ThingsBoard Cloud.

5.6 Các service bắt buộc

- mosquitto — MQTT broker.
- sensor-bin-01/02/03 — sensor mỗi thùng.
- actuator-bin-01/02/03 — actuator mỗi thùng.
- waste-gateway — edge IoT gateway.
- influxdb — time-series database.
- grafana — dashboard edge.
- waste-api — REST API (FastAPI).

Phải đủ các nhóm chức năng: sensor simulator, actuator simulator, broker, gateway, database, dashboard, API.

5.7 Thiết kế MQTT topic

```

waste/bin-01/sensor/telemetry      # fill/weight/gas/temp from bin sensor
waste/bin-01/actuator/command      # command to lock / compactor / buzzer / dispatch
waste/bin-01/actuator/status       # actuator status feedback
waste/bin-01/gateway/normalized    # normalized data
waste/bin-01/gateway/event         # event raised by gateway
  
```

5.8 Message format yêu cầu

Telemetry từ sensor:

```

{
  "device_id": "sensor-bin-01", "area_id": "district-1", "bin_id": "bin-01",
  "fill_level": 88.0, "weight_kg": 42.5, "methane_ppm": 350.0,
  "temperature": 33.0, "lid_status": "closed", "tilt": false,
  "timestamp": "2026-06-10T10:00:00Z"
}
  
```

Command từ gateway tới actuator (target ∈ {lock, compactor, buzzer, dispatch}):

```

{ "bin_id": "bin-01", "target": "dispatch", "action": "on",
  "reason": "bin_full", "timestamp": "2026-06-10T10:00:05Z" }
  
```

Status từ actuator:

```

{ "device_id": "actuator-bin-01", "bin_id": "bin-01",
  "lock": "off", "compactor": "off", "buzzer": "off", "dispatch": "on",
  "last_command_reason": "bin_full", "timestamp": "2026-06-10T10:00:06Z" }
  
```

Event từ gateway:

```
{ "bin_id": "bin-01", "event_type": "bin_full", "severity": "warning",
  "value": 88.0, "threshold": 85.0, "action_taken": "dispatch_on",
  "timestamp": "2026-06-10T10:00:05Z" }
```

5.9 Yêu cầu lập trình

Virtual sensor

1. Đọc env: AREA_ID, BIN_ID, DEVICE_ID, MQTT_BROKER, PUBLISH_INTERVAL.
2. Mô phỏng mức đầy tăng dần theo thời gian (rác bỏ vào), khối lượng tăng theo; giảm về 0 khi được thu gom. Có xu hướng, không random độc lập.
3. Sinh tình huống: thùng đầy, nhiệt độ cao (nguy cơ cháy), khí methane tăng, thùng bị đổ nghiêng.
4. Publish JSON lên waste/<bin>/sensor/telemetry.

Virtual actuator

1. Subscribe topic command của thùng; parse JSON.
2. Cập nhật trạng thái lock, compactor, buzzer, dispatch.
3. Publish lại trạng thái lên topic status; ghi log.

Edge IoT gateway

1. Subscribe telemetry tất cả thùng; validate và normalize.
2. Lưu trạng thái mới nhất từng thùng; duy trì **danh sách thùng cần thu gom**.
3. Ghi telemetry vào InfluxDB; chạy rule engine; sinh và publish event.
4. Gửi command tới actuator; đẩy telemetry lên ThingsBoard và nhận RPC (mục 5.10).

Rule engine

Tối thiểu 4 luật:

1. fill_level > 85 ⇒ dispatch=on, thêm vào danh sách thu gom; event bin_full, severity warning. > 95 severity critical.
2. temperature > 60 ⇒ buzzer=on, **cấm** compactor; event fire_risk, severity critical.
3. methane_ppm > 500 ⇒ event gas_alert, severity warning (gợi ý mở nắp thông khí).
4. weight_kg > 60 ⇒ lock=on (chống quá tải); event overweight, severity warning.
5. (Mở rộng) tilt=true ⇒ event bin_tilted (bảo trì); sensor offline ⇒ sensor_offline.

REST API

GET /health	GET /bins
GET /bins/{bin_id}/state	GET /bins/{bin_id}/events
GET /collection/route	# list of bins that need collection now
POST /bins/{bin_id}/command	# body: {"target":"compactor","action":"on","reason":"manual"}

5.10 Tích hợp ThingsBoard (uplink + RPC)

1. Tạo thiết bị Gateway, đặt token vào TB_GATEWAY_TOKEN; connect mỗi thùng rồi đẩy telemetry:

```
// publish -> v1/gateway/telemetry
{ "bin-01": [{"ts":1750000000000,"values":{"fill_level":88,"methane_ppm":350,"temperature":33}}],
  "bin-02": [{"ts":1750000000000,"values":{"fill_level":40,"methane_ppm":120,"temperature":30}}] }
```

2. Nhận RPC để điều khiển nén rác/khóa nắp từ trung tâm:

```
// receive <- v1/gateway/rpc
{ "device": "bin-01", "data": { "id": 4, "method": "setCompactor", "params": true } }
// reply -> v1/gateway/rpc : { "device": "bin-01", "id": 4, "data": { "success": true } }
```

3. Trên ThingsBoard: dashboard bản đồ thùng rác (màu theo mức đầy), gauge mức đầy, bảng thùng cần thu gom; nút RPC setCompactor/setLock/setBuzzer; Alarm khi bin_full hoặc fire_risk cấu hình bằng Rule Chain.

5.11 Yêu cầu lưu trữ InfluxDB

- bin_telemetry — tag: area_id, bin_id; field: fill_level, weight_kg, methane_ppm, temperature.
- gateway_events — tag: bin_id, event_type, severity; field: value, threshold.
- actuator_status — tag: bin_id; field: lock, compactor, buzzer, dispatch.

5.12 Yêu cầu Grafana Dashboard

1. Mức đầy theo thời gian cho từng thùng (kèm đường ngưỡng thu gom).
2. Khối lượng theo thời gian cho từng thùng.
3. Nhiệt độ và methane theo thùng (cảnh báo cháy/khí).
4. Trạng thái khóa/nén/báo động/điều xe theo thùng.
5. Số event (bin_full, fire_risk) theo thời gian.
6. Bảng thùng cần thu gom hiện tại và bảng event gần nhất.

5.13 Docker và ảo hóa

Tuân thủ mục 4–5 Khung yêu cầu chung: mỗi service tự lập trình có Dockerfile riêng; volume cho InfluxDB/Grafana; cấu hình qua .env; service gọi nhau bằng tên.

5.14 Câu hỏi bắt buộc trong báo cáo

1. Vì sao thu gom *theo nhu cầu* hiệu quả hơn lịch cố định? Edge gateway hỗ trợ thế nào?
2. Nhóm thiết kế topic và message format cho thùng rác thế nào?
3. Rule engine xử lý đầy/cháy/quá tải theo những luật nào?
4. Gateway duy trì danh sách thùng cần thu gom ra sao?
5. Gateway đóng vai trò ThingsBoard Gateway như thế nào?
6. Vì sao một số quyết định (báo cháy) nên xử lý ngay tại edge?
7. Vì sao container không nên dùng localhost để gọi service khác?
8. Muốn tối ưu lộ trình thu gom toàn thành phố trên ThingsBoard cần thêm gì?

5.15 Phân công gợi ý cho nhóm 3 sinh viên

- **SV1:** virtual sensor, virtual actuator, thiết kế topic/message.
- **SV2:** edge gateway, rule engine, danh sách thu gom, ghi InfluxDB.
- **SV3:** tích hợp ThingsBoard + dashboard + alarm, REST API, Docker Compose, Grafana, README.

5.16 Tiêu chí chấm điểm

Nội dung	Điểm
Kiến trúc Virtual Waste Gateway hợp lý	1.0
Thiết kế MQTT topic & message format rõ ràng, mở rộng được	1.0
Virtual sensor mô phỏng mức đầy/khối lượng có xu hướng và tình huống đặc biệt	1.0
Virtual actuator (khóa/nén/còi/điều xe) nhận command và publish status	0.8
Gateway validate, normalize, duy trì danh sách thu gom, lưu trạng thái	1.2
Rule engine xử lý đầy/cháy/quá tải và điều khiển tự động	1.0
Tích hợp ThingsBoard: telemetry gateway + RPC điều khiển từ xa	1.2
Ghi telemetry/event/status vào InfluxDB	0.8
Grafana & ThingsBoard dashboard thể hiện rõ mức đầy, event, trạng thái	0.8
Docker Compose, Dockerfile, network, volume, env đúng	1.0
README, báo cáo, demo và giải thích hệ thống	0.7 + 0.5
Tổng	10.0

5.17 Yêu cầu nâng cao (cộng điểm khuyến khích)

1. Tối ưu lộ trình thu gom (gợi ý thứ tự thùng) từ danh sách cần thu gom.
2. Bản đồ thùng rác theo mức đầy (màu) trên ThingsBoard.
3. Shared Attributes để chỉnh ngưỡng fill_level từ xa.
4. Health check cho các service trong Compose.
5. Alarm + thông báo khi fire_risk hoặc thùng tràn.
6. Dự báo thời điểm thùng đầy từ tốc độ tăng mức đầy.
7. Unit test cho rule engine.
8. Cơ chế reconnect khi mất kết nối MQTT/ThingsBoard.

5.18 Kết luận

Đề tài giúp sinh viên xây dựng hệ thống quản lý rác thải ảo hóa hướng tới thu gom thông minh theo nhu cầu, kết hợp xử lý cục bộ tại biên (an toàn cháy nổ, duy trì danh sách thu gom) với vận hành tập trung trên ThingsBoard Cloud. Sinh viên vận dụng đầy đủ MQTT, rule engine, InfluxDB, Grafana, ThingsBoard Gateway API, RPC, REST API và Docker Compose trong một bài toán đô thị thông minh có tính ứng dụng cao.